

AudioPlatform

for Teensy 4.1

apApp User Guide



Revision 1.0.1, July 31, 2025

Donald Swearingen and ProtoSupplies

Introduction

First and foremost, AudioPlatform for Teensy 4.1 is a hardware platform for experimentation and development of simple-to-sophisticated, stand-alone, audio systems and applications, built within and upon the Teensy4.1/TeensyDuino/Arduino environment.

AudioPlatform is a response to the difficulty that many potential creators of Teensy-based audio systems face: the challenge of acquiring or themselves building some sort of hardware, often a “DIY” PCB board and list of components, to accommodate their projects and explorations. Many of those with such interests simply lack the skills or resources to complete this process; and the general lack of availability of off-the-shelf hardware suitable to their projects restricts their options to the use of breadboards and/or simply-wired “perf” boards to do so, not to mention creating modern PCB-based devices with surface mounted components.

This was pretty much the case for me, as well. Although I’m a life-long professional musician and composer, I’ve also over the years acquired considerable skills in software and electronic systems development, creating software and hardware components and systems in support of numerous live musical and artistic projects. Most of that work had also involved the use of powerful computers.

AudioPlatform aims to address that deficit by offering a system that includes a significant set of resources for audio applications. These include both hardware and software components.

AudioPlatform Hardware

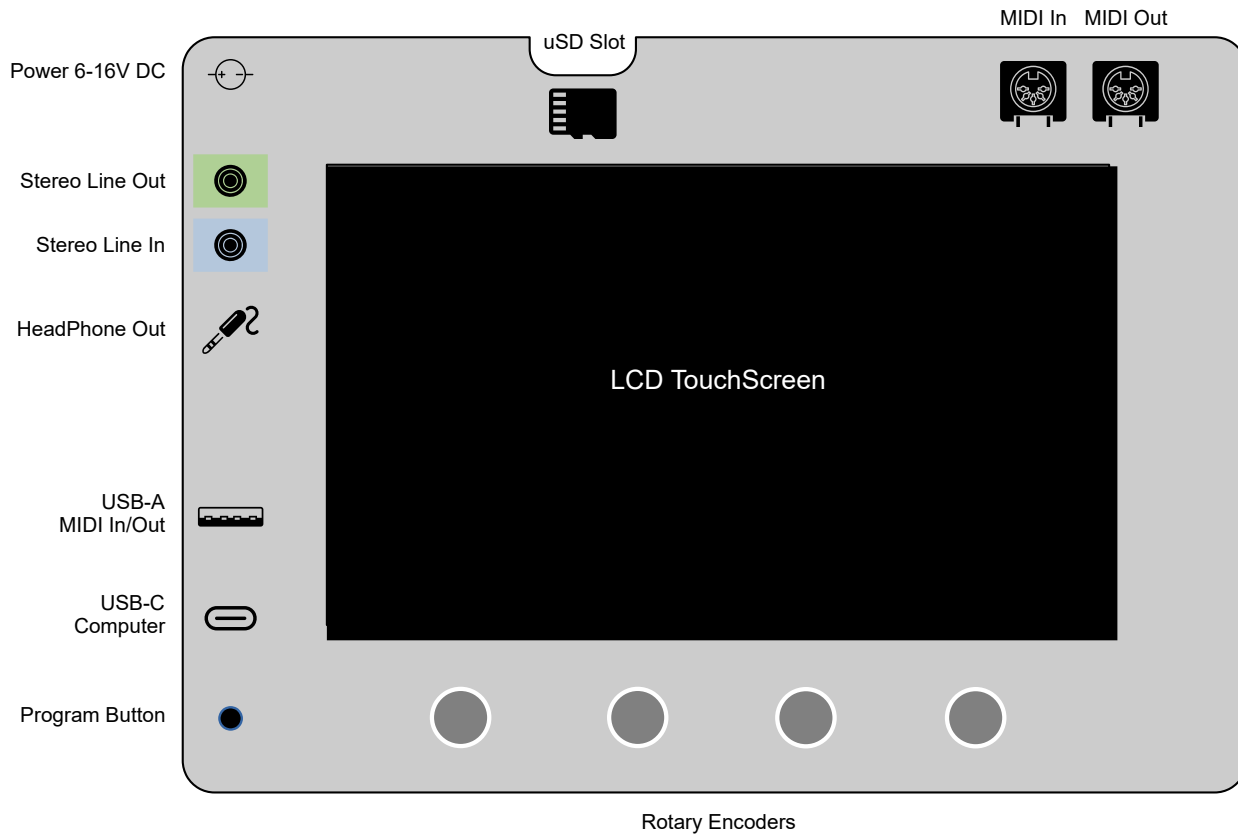
- Black Acrylic-Matte Housing and Base
- 800x480 Pixel RGB LCD Capacitive Touch Screen
- Teensy 4.1 @600 MHz with 1 MB RAM
- SGTL5000 Audio Codec
- 8 MB Non-Volatile Program Flash
- 16 MB PSRAM
- 128 MB Non-Volatile Serial Flash Storage
- Teensy 4.1 SD Card Reader, up to 32GB
- USB I/O on both MIDI Host and USB-C Connections
- Midi I/O on both USB and MIDI DIN Ports
- Audio Line In/Out on 3.5 mm Female Connectors
- Audio In/Out over USB
- Headphone Out on 3.5mm Female Connector
- 4 Rotary Encoders for Menu and Device Parameter Settings

AudioPlatform Software

The open-source AudioPlatform software application, named simply *AudioPlatform* and referred to in documentation as **apApp**, is installed and shipped with the AudioPlatform hardware. ApApp features include:

- Five Multi-Voice Sound Players: Dexed DX7 Emulator, SD WAV File Player, Serial Flash RAW File Player, Analog-Style Synthesizer, Stereo Audio Input
- 512 Dexed Voices in 16 Banks of 32 Voices, Including All Voice Banks of the Original DX7; 10 Banks Stored in Non-Volatile Program Memory; 6 User Banks Loadable from SD Card at Startup; Fully Compatible with All Banks Created in Dexed PC Application and Other Sources (1000s of Banks Available on Various WebSites)
- Stereo Reverb and Stereo Ping-Pong Delay
- All Players and Effects Simultaneously Active or Inactive Based on Preset Settings (no Mutually-Exclusive Devices)
- Player and Effects Parameters Stored in Recallable Preset Banks, 32 Presets per Bank, Unlimited Bank Storage on microSD Card with Instant Recall
- Flexible System Architecture with Individual Components (Players/Effects, Other Elements) Modifiable and/or Replaceable
- Extensive Display and System Support with C++ Libraries, Many of Which Are Application-Independent
- TouchScreen and Encoder Support for Screen Selection, Editing and Selection of Player, Effects, and SystemParameters
- Real-Time Clock Support
- Real-Time Display of Audio Levels, CPU and Memory Usage
- User Utilities for File Viewing and Management

Input/Output Connections



Connection Details

Power 6-16V DC

Power is delivered through the DC input connector from a standard 9VDC wall adapter with a center-positive (+), 2.1mm female connector. Note that power supplied through the Teensy 4.1 USB-C connection to a computer is disabled---AudioPlatform can only be powered from the DC input.

Ken: Normally ship with a 7.5V adapter

Stereo Line In/Out

Stereo Line inputs and outputs are female 3.5mm TRS connectors.

Ken: Maybe worth calling out SGTL5000 audio subsystem

HeadPhone Out

The headphone output is a 3.5mm TRS female connector.USB-A Connector

USB-A Connector

The USB-A connector is used for MIDI input from a MIDI keyboard or controller, and can supply power to a connected MIDI device.

Ken: Up to a maximum of 500mA. Higher current devices need to be externally powered.



In the first release of the AudioPlatform application (apApp), only MIDI input is supported. Support for MIDI output, and multiple input devices via a USB Hub are planned for future versions.

USB-C Connector

The USB-C connection is used exclusively for connection of AudioPlatform to a computer and supports both the Arduino IDE, and Serial IO.



Support for MIDI and Audio IO via the USB-C connection is planned for a future release of the AudioPlatform application.

Program Button

The small program button is employed in the process of downloading a pre-compiled executable (".exe") image to the AudioPlatform's Teensy 4.1, when using the Teensy Loader application (an independent component of the Arduino/Teensy environment which allows downloading independent of whether or not the Arduino IDE is active).

microSD Card Slot

The microSD (uSD) slot accepts a standard microSD card. Though AudioPlatform can start up and perform many of its functions without the presence of a uSD card, there are numerous features and capabilities of the apApp that depend on and require a card to be present. The types and compatibilities of various uSD media are discussed further in **SECTION REFERENCE**.

Ken: 32GB was the limitation. Now in theory can support up to 2TB but I haven't tried anything other than 32GB which is what I ship with

<https://forum.pjrc.com/index.php?threads/microsd-card-max-size-in-teensy-4-1.67242/>

Paul StofReggen 5/19/21

You need to use SdFat or the latest 1.54 betas (where SD is now just a thin wrapper for SdFat) for cards larger than 32GB.

If you use Teensyduino 1.53 or older, the old Arduino SD library supports only up to 32GB. If you're not sure which version you have, in Arduino click Help > About (or Teensyduino > About on Macintosh) to see which version is installed.

Paul StofReggen 10/17/22

Use FAT for 32GB and smaller cards, or exFat for larger than 32GB.

Theoretically up to 2TB should work, but as far as I know 1TB is the largest card anyone has tried. The largest card I own is 512GB. It works fine.

KurtE 10/17/22

Note: with SD library usage of SDFat, there are different classes you can setup to use:

FatLib - which does: Fat16, Fat32 and optionally Fat12

ExFatLib - which does ExFat

FsLib - which is setup to use Fat or ExFat depending on the card.

Our SD Library uses the FsLib (at least for most teensy boards)

We have format code built in, including in some of our sketches.
However, you should be able to use any formatter on your PC.

I know at least at one point the developer of SDFat recommended the formatter
from: <https://www.sdcard.org/downloads/formatter/>
Which should initialize and format the cards to the SD standards...

MIDI DIN In/Out

MIDI input is also possible on the 5-pin DIN connector (original MIDI standard still used with numerous current MIDI devices).



Support for MIDI output, and simultaneous use of both the MIDI DIN connection, and the USB-Host MIDI IO connection are planned for apApp.

Rotary Encoders

The 4 rotary encoders are an important and integral component of the AudioPlatform design. In the AudioPlatform application, the encoders are used to enter settings and parameters of the various devices implemented in the application. Detailed descriptions of encoder usage are outlined in **SECTION REFERENCE**.

LCD Screen

Ken: Probably worth also calling out 7" RA8875 on LCD

Documentation Ideas and Plans

Documentation Topics

- Intent/Audience
- Description
- Designer Motivations
- Deliverables
 - SD Contents
 - SD Data Maintenance Recommendations
 - Serial Flash Contents
 - Program Flash Non-Volatile System Data
- Screens and User Interface
- AudioPlatform Application Basics

AudioPlatform Documents

- Introduction
- Getting Started Guide
- User's Guide
- Application and Programmer's Reference

Introduction

Section/Topic Ideas

- What is AudioPlatform?
- Background
-

Getting Started Guide

Section/Topic Ideas

- Installation and Hookup
- Verification
- System Data SD and Flash Based
- SD and Flash Data Contents in Shipped System
- SD File Backup and Maintenance on PC (User Organization)
- Basic Menus
- Players
- Efx and MIDI Devices
- Generic MIDI Controller (as shipped) *Clarify: Compiled for generic; product does not include MIDI controller*
- Limitations (subject to future updates; ex: run-time MIDI selection)
- Instructions on how to load EXEs for other controllers and configurations
- Presets
- PresetsLive
- Screens

Ken: Should have a list of what is in the box such as:

7.5V AC/DC Adapter?

32GB MicroSD Card?

MicroSD to SD adapter?

USB-C cable?

Tilting base?

Hardcopy getting started guide?

AudioPlatform User's Manual

Section/Topic Ideas

- System Overview
- Presets and Preset (Data) Manager
 - Presets Basics
 - Preset Structures (w/Diagrams)
 - Presets and Data Management
 - Data Flow
 - Preset Devices
- Basic Menus
- Player Devices
 - Player Screens
 - Player Parameters (Common and Individual)
- Audio and MIDI (Input/Controller) Devices
 - Player Screens
 - Player Parameters (Common and Individual)
- MIDI Controlllers
- Presets
- PresetsLive
- Screens

AudioPlatform Application and Programmer's Reference

Section/Topic Ideas

- Design Principles and Goals
- Application and System Architecture (Diagrams)
- Application Elements
 - Primary Application Components
 - Application-Specific Classes (Screen and Support Classes)
 - Application-Independent Classes
 - Utility Classes
- Application Components
 - AudioPlatorm
 - Presets (To become StateMgr)
 - Screens
 - AudioDevices
 - InputDevices
- Presets/Data Management
- Arduino Mixed C/C++ Environment
 - One way function calls/callbacks C++ to C functions, but not C to C++ class method
 -

Deliverables

microSD Card

Although the AudioPlatform application can be operated without the presence of a microSD card (generally referred to in this document as the SD Card or μ SD) in the Teensy 4.1 card slot, there are a number of features of the system that are dependent on the presence of the card and its contents. These include:

- Access to Stereo WAV (".wav") audio files located on the SD card for playback by the SD Wav Player
- Access to banks of Monaural (Mono) RAW (".raw") audio files to be transferred to the internal Serial Flash storage for playback by the Flash Raw Player
- Access to user Preset Banks (".pbk") files stored on the card
- Access to user-defined Voice Banks (".syx") for playback using the FM Synthesizer

Application Versions and Options

As discussed in detail in the *AudioPlatform User's Manual* and the *AudioPlatform Application and Programmer's Reference*, there are a number of application options that can be configured only by making modifications to the application source-code, and recompiling (rebuilding the application using the Audio IDE). However, so that users may explore and use the application without the need to set up the Arduino IDE and application software environment and carry out this process, the system is shipped with a number of pre-built "EXE" modules that can be easily downloaded into AudioPlatform without going through this process.

The 2 most important selections for getting started are 1) the selection of a generic or specific MIDI controller, and 2) whether the controller is to be connected to AudioPlatform via a USB connection, or via a standard MIDI "DIN" connection (5-pin round connector on both ends).

Default Options

- Generic MIDI Controller
- MIDI Connection via USB

Alternate Options

Alternate options can be employed both for the MIDI controller and the controller connection. Separate, installable EXE files are available for download from the link

<https://www.UrlLinkForAlternateEXEs.com>

Included are EXE modules for the following setups:

- Generic MIDI Controller, MIDI connection via USB
- Generic MIDI Controller, MIDI connection via DIN input
- Arturia Minilab3 MIDI Controller, MIDI connection via USB
- Arturia Minilab3 MIDI Controller, MIDI connection via DIN input
- Novation LaunchKey 25 MIDI Controller, MIDI connection via USB. *Note: LaunchKey 25 only includes a USB MIDI output. DIN connections are not available.*
- Novation LaunchKey 49 MK3 MIDI Controller, MIDI connection via USB
- Novation LaunchKey 49 MK3, MIDI connection via DIN input

Installing Alternate Options

Teensy Loader Application

Pre-built Teensy EXE executable files can be easily downloaded to AudioPlatform (or any compatible Teensy device connected to a computer via its standard USB connection) using the *Teensy Loader* application. Versions of Teensy Loader are available for Windows, MacOS, and Linux at the following locations:

Windows

https://www.pjrc.com/teensy/loader_win10.html

MacOS

https://www.pjrc.com/teensy/loader_mac.html

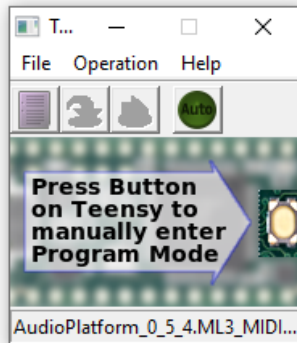
Linux (Ubuntu)

https://www.pjrc.com/teensy/loader_linux.html

Using Teensy Loader

Once you've downloaded the Teensy Loader and a suitable alternate EXE for AudioPlatform:

1. double click on the loader (Teensy.exe) to display its small screen:



2. Click on the File menu
3. Navigate to the location of the EXE to be installed and click to select (the name of the selected file will be displayed at the bottom of the loader window, as above)
4. Press the PROGRAM button at the bottom left and beneath the top panel of AudioPlatform to begin the download.

Once the download is completed, AudioPlatform will restart and display its Main Menu screen.



Among the options on the list for inclusion in a future version of the apApp are run-time selection of the MIDI controller and the connection port (USB or MIDI). In the case of the port connection, the intention is that, in the future, both ports will be active, although only a single controller may be recognized (another potential addition is the ability to use more than one controller, and perhaps a USB hub).

SD Contents

Stereo WAV Files

Bank 0

000-SoundFile.wav
001-SoundFile.wav
002-SoundFile.wav
003-SoundFile.wav
004-SoundFile.wav
005-SoundFile.wav
006-SoundFile.wav
007-SoundFile.wav
008-SoundFile.wav
009-SoundFile.wav
010-SoundFile.wav
011-SoundFile.wav
012-SoundFile.wav
013-SoundFile.wav
014-SoundFile.wav
015-SoundFile.wav
016-SoundFile.wav
017-SoundFile.wav
018-SoundFile.wav
019-SoundFile.wav
020-SoundFile.wav
021-SoundFile.wav
022-SoundFile.wav
023-SoundFile.wav
024-SoundFile.wav

Bank 1

100-SoundFile.wav
101-SoundFile.wav
102-SoundFile.wav
103-SoundFile.wav
104-SoundFile.wav
105-SoundFile.wav
106-SoundFile.wav
107-SoundFile.wav
108-SoundFile.wav
109-SoundFile.wav
110-SoundFile.wav
111-SoundFile.wav
112-SoundFile.wav
113-SoundFile.wav
114-SoundFile.wav
115-SoundFile.wav
116-SoundFile.wav
117-SoundFile.wav
118-SoundFile.wav
119-SoundFile.wav
120-SoundFile.wav
121-SoundFile.wav
122-SoundFile.wav
123-SoundFile.wav
124-SoundFile.wav

Stereo WAV Files (continued)

Bank 2

200-
201-
202-
203-
204-
205-
206-
207-
208-
209-
210-
211-
212-
213-
214-
215-
216-
217-
218-
219-
220-
221-
222-
223-
224-

Bank 3

300-
301-
302-
303-
304-
305-
306-
307-
308-
309-
310-
311-
312-
313-
314-
315-
316-
317-
318-
319-
320-
321-
322-
323-
324-

Mono RAW Files

Bank 0

000-
001-
002-
003-
004-
005-
006-
007-
008-
009-
010-
011-
012-
013-
014-
015-
016-
017-
018-
019-
020-
021-
022-
023-
024-

Bank 1

100-
101-
102-
103-
104-
105-
106-
107-
108-
109-
110-
111-
112-
113-
114-
115-
116-
117-
118-
119-
120-
121-
122-
123-
124-

Mono RAW Files (continued)

Bank 2

200-
201-
202-
203-
204-
205-
206-
207-
208-
209-
210-
211-
212-
213-
214-
215-
216-
217-
218-
219-
220-
221-
222-
223-
224-

Bank 3

300-
301-
302-
303-
304-
305-
306-
307-
308-
309-
310-
311-
312-
313-
314-
315-
316-
317-
318-
319-
320-
321-
322-
323-
324-

SD Card Usage and Management

SD Card Usage and Management

Although the presence of an SD card is not required to use many of the basic features of apApp, the full extent of application capabilities is only possible when an SD card is present in the Teensy 4.1 microSD slot.

Because of this and various other considerations, it is considered fundamental to the full benefits of AudioPlatform that the contents of the SD card be managed in a well-organized and careful manner.

Other than the capability to name, store, and load preset banks to/from the sdCard, apApp makes no provision of sdCard file and directory management. It is thus necessary that users organize and

manage soundfile resources and SD card contents on a PC (Windows, Mac, Linux) using various available applications and/or utilities. The renaming of a collection of audio files to conform to the naming requirements for SD card audio files in apApp is perhaps the primary issue in this connection, and it is necessary to use some appropriate means of doing so. There are numerous paid and free applications that can be used for this purpose. Examples include:

PC-Based File Management Utilities

Windows

- Microsoft PowerToys PowerRename
- Advanced Renamer
- Bulk Rename Utility
- File Renamer
- Metamorphose 2

MacOS

- Finder's Built-in Feature
- Transnomino
- NameChanger
- Metamorphose 2

Linux

- GPRename
- pyRenamer
- Thunar
- Metamorphose 2

SD Card Format

SD Card Content

File Naming

In order to be recognized in apApp, WAV and RAW audio filenames must follow a specific convention that indicates which of the 4 sample banks the file is to be located in, and which MIDI note number (0-99, 100-127 ignored) will play the sample.

The convention dictates that the sample filename must begin with a 3-digit number (leading 0s required) followed by a dash '-' and the filename, and ending with the suffix .wav or .raw as the case may be. The 3 digits indicate both the bank number:

- 000-099 Bank 0, MIDI notes 0-99
- 100-199 Bank 1, MIDI notes 0-99
- 200-299 Bank 2, MIDI notes 0-99
- 300-399 Bank 2 MIDI notes 0-99

Examples

Bank 0

- 000-Note0_Sample.wav
- 060-MiddleC_Sample.wav
- 099-Note99_Sample.wav

Bank 2

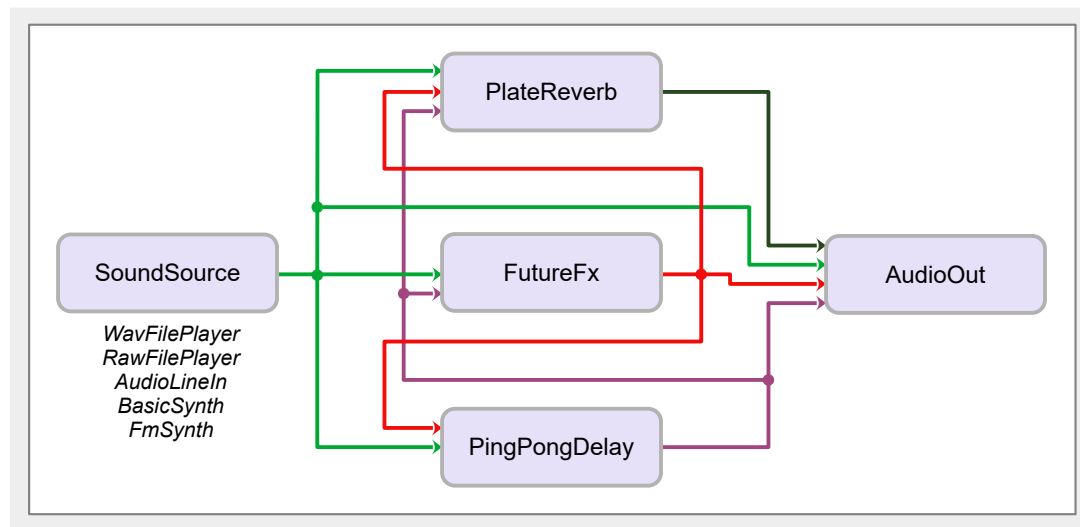
- 250-SoundEffect_01.wav
- 260-SoundEffect_02.wav
- 299-HighVoice.wav

The same conventions are used for RAW audio file banks stored in non-volatile Serial Flash.

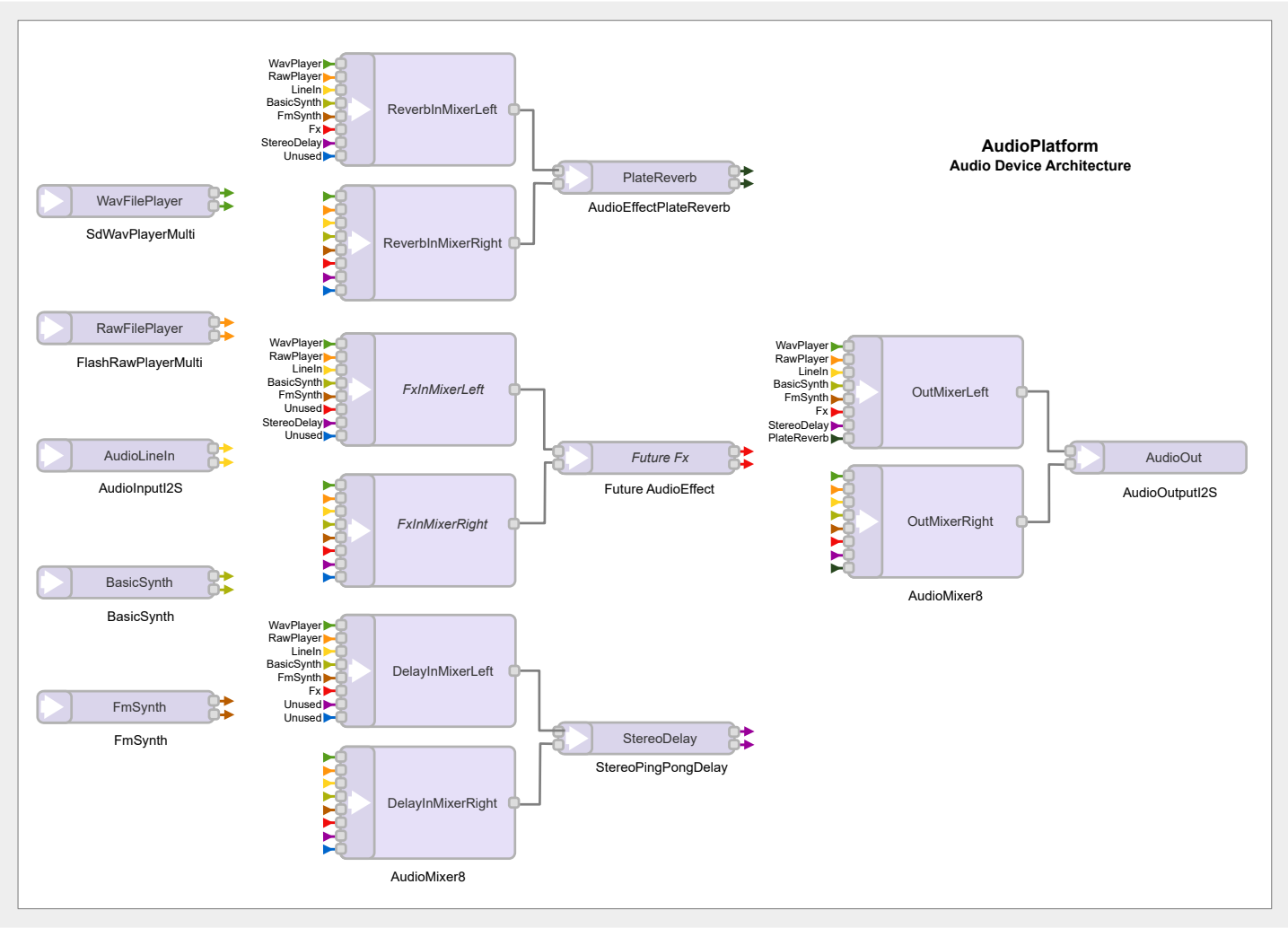
Filename Management

In general, the source of most user-supplied audio WAV and Raw files will be

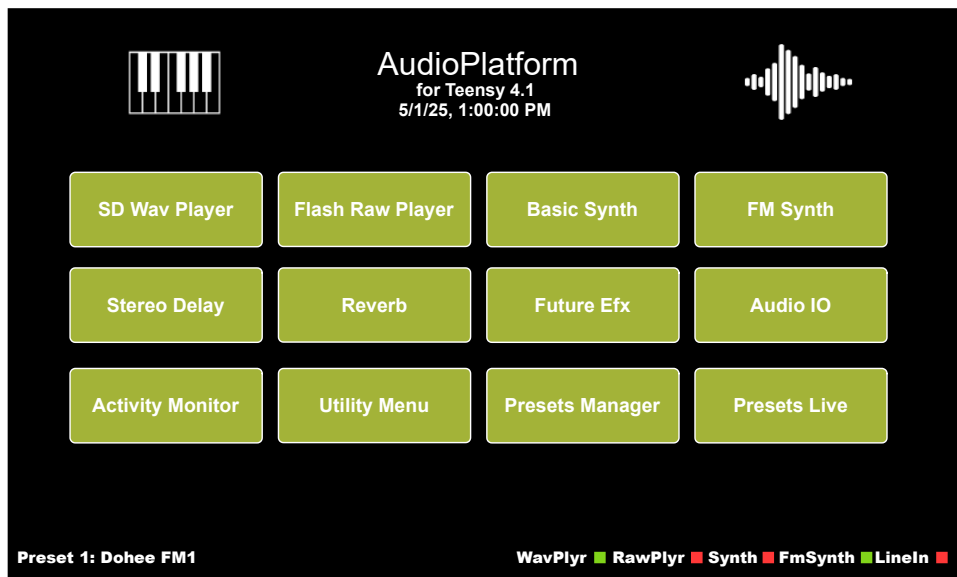
Signal Flow



Connection Detail



Main Menu



Main Menu Screen Types

SD Wav Player

Flash Raw Player

Basic Synth

FM Synth

Stereo Delay

Reverb

Future Efx

Audio IO

Activity Monitor

Utility Menu

Presets Manager

Presets Live

ParamButton Screens

Support Screens

AudioPlatform

for Teensy 4.1

5/1/25, 1:00:00 PM

WavPlyr

RawPlyr

Synth

FmSynth

LineIn

Player and Audio Devices

SD Wav Player

Flash Raw Player

Basic Synth

FM Synth

Stereo Delay

Reverb

Future Efx

Audio IO

Activity Monitor

Utility Menu

Presets Manager

Presets Live

Player Devices

Audio Devices

AudioPlatform

for Teensy 4.1

5/1/25, 1:00:00 PM

WavPlyr

RawPlyr

Synth

FmSynth

LineIn

Player Devices

Player Devices occupy the topmost row of the Main Menu screen. apApp features 4 players, which play audio media in response to inputs from an external MIDI device, and each of which operates on a separate media type:

- The *SD Wav Player* plays Stereo WAV formatted files located on the Teensy 4.1 SD card
- The *Flash Raw Player* plays Monaural (single channel, headerless) audio files located in the AudioPlatform's 128 MB non-volatile Flash Memory
- The *Basic Synth* is an implementation of a basic digital audio synthesizer with 12-voice polyphony
- The *FM Synth* is a Teensy Audio System implementation of the micro-Dexed synthesizer which is an emulator of the classic DX7 synthesizer, featuring 8-voice polyphony

Audio Devices

Audio Devices occupy the middle row of the Main Menu screen. apApp features 4 audio devices, which play various roles in the apApp audio architecture:

- The *Stereo Delay* device implements a stereo “ping-pong” digital delay, with up to 3 seconds of delay per channel, and both individual channel and cross-channel feedback paths
- The *Reverb / EFX* device features a Plate Reverberation unit, and a second effects unit pathway intended for addition of a 3rd audio effect in future apApp releases.
- The *Audio IO* device provides for control of the levels and effects settings of the system audio paths, with settings for Input and Output levels, effects send levels, and other audio system parameters
- The *MIDI* device features settings related to a connected MIDI control device, such as MIDI channel and destination player settings

Support and Utility Devices

Support Devices occupy the bottom row of the Main Menu screen, and serve as a gateway to additional screens and menus that perform various system and user support functions.

- The *Activity Monitor*
- The *Utility Menu*
- The *Presets Manager*
- The *Presets Live* device

Screens and Settings

Screen Icons

Screen icons (*buttons*) appear on all screens, and serve to allow the user to navigate among application screens (*Navigation Icons*), and to make selections or perform actions on the currently displayed screen (*Action Icons*).

Navigation Icons

Home



This button appears in the upper-right corner of all level 2 screens (screens one level below the main menu). Touching the button returns the user to the Main Menu screen.

Back



This button appears in the upper-right corner of level 3 and greater screens (screens at least two levels below the Main Screen).

Enter



This button may appear in various screens to provide access to a lower-level screen that is only accessible from the current screen.

Exit



This button appears in screens accessed using the Enter button, allowing the user to return to the screen which invoked the currently displayed screen.

Action Icons

All Notes Off



This button appears at the top of the Presets Live screen. When pressed, all notes currently playing are silenced. (Note: this function can also be assigned to a button or other control on a connected MIDI keyboard or controller).

Ignore Note Off



This button appears at the top of the Presets Live screen. When pressed, all notes currently held or played thereafter continue to play, ignoring key or sustain pedal releases. (Note: this function can also be assigned to a button or other control on a connected MIDI keyboard or controller).

Save Changes



This button appears at the top of all ParamButton screens. The text label of the button “Save Changes” normally appears in white. If any parameter changes are made to the displayed device using a rotary encoder, the text changes to red “**Save Changes**”, and remains so until the button is touched, at which point it returns to its normal white appearance. This process is discussed in detail in the **SECTION REFERENCE**.

Copy Settings



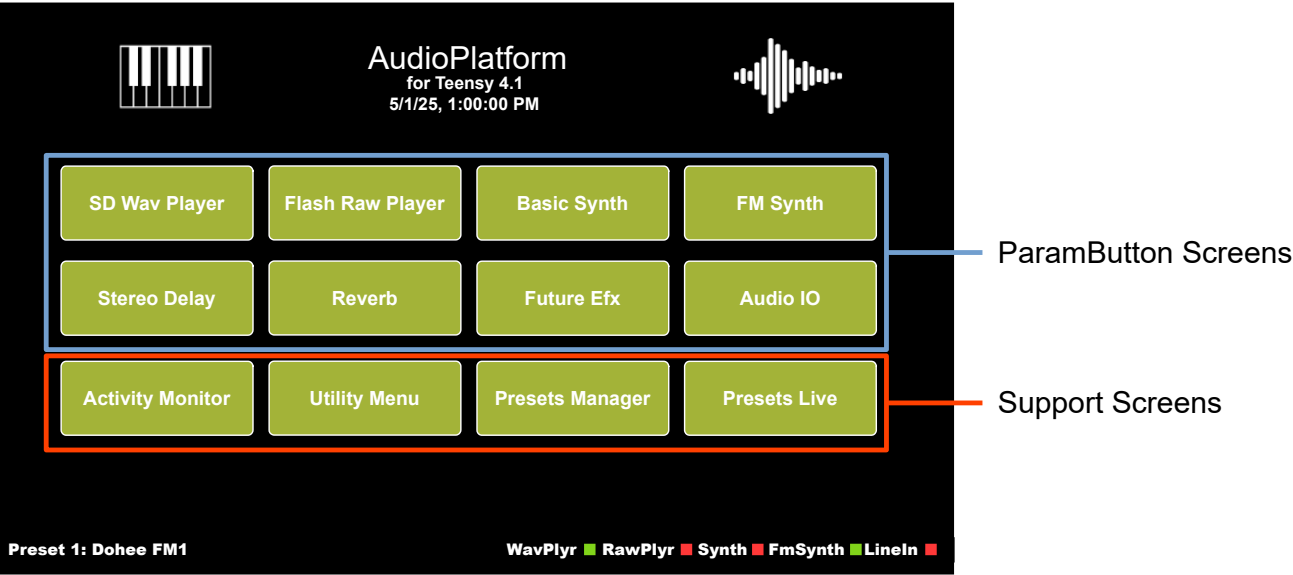
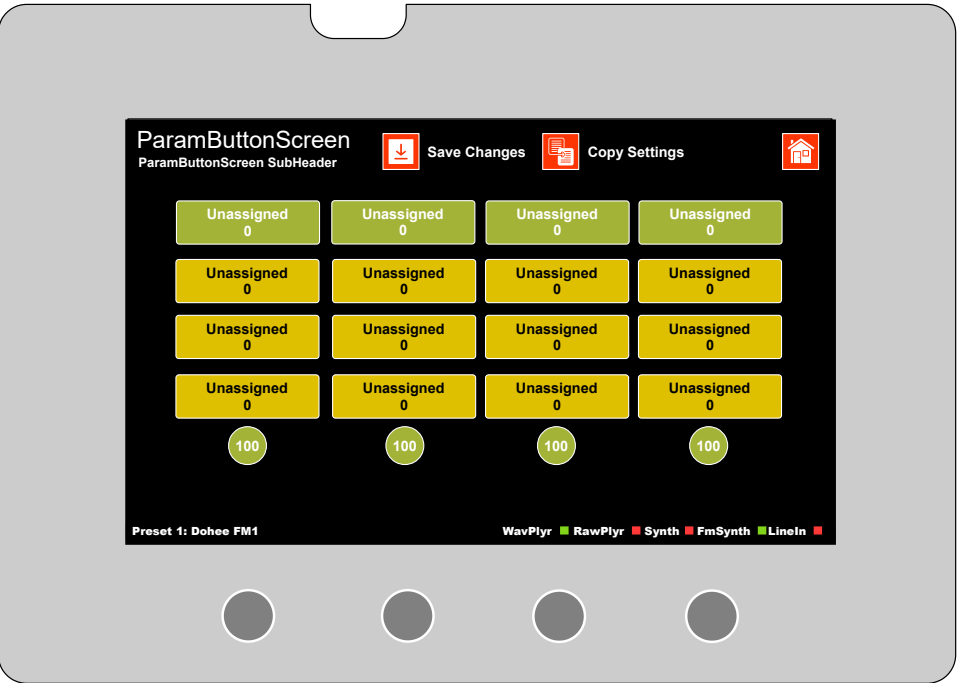
This button appears on all ParamButton screens next to the Save Changes button. When pressed, it displays a screen with buttons representing the presets currently loaded in the *Import* presets bank. When one of the import preset buttons is touched, the device settings of the import device are copied to the currently displayed device. This process is discussed in detail in the **SECTION REFERENCE**.

Scroll Up Down

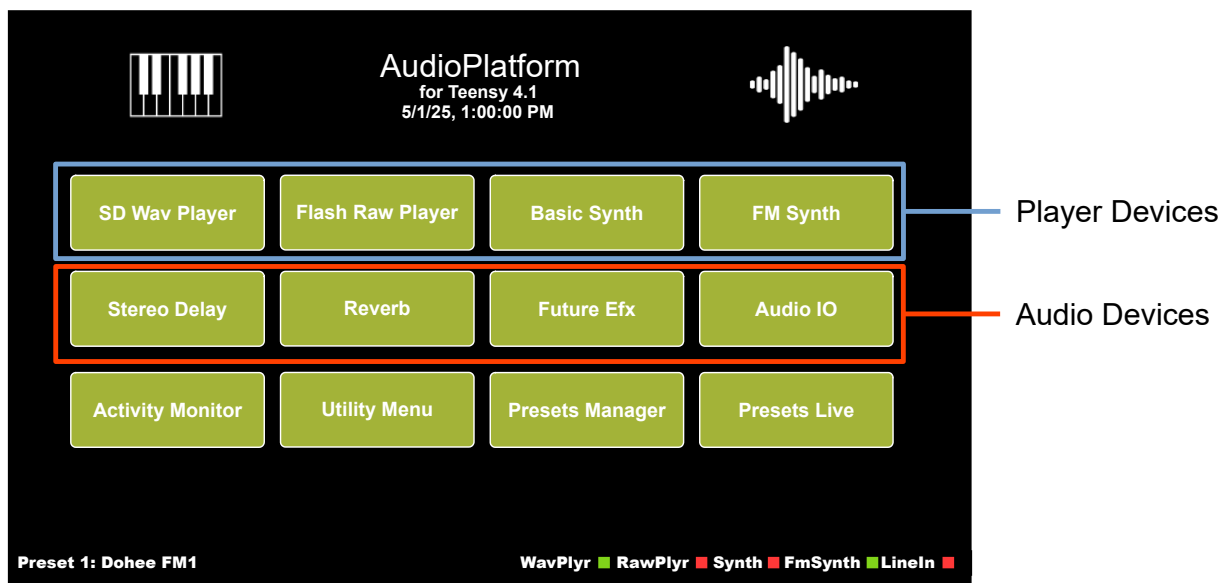


These buttons appear on screens that display lists of files or other names either for review or selection of items (for example, a preset bank file on the SD card). When simply displaying lists, the screen will scroll up or down repeatedly until the button is released. When displaying items for selection, repeated touches are required to scroll the list.

ParamButton Screens



Player Device Screens



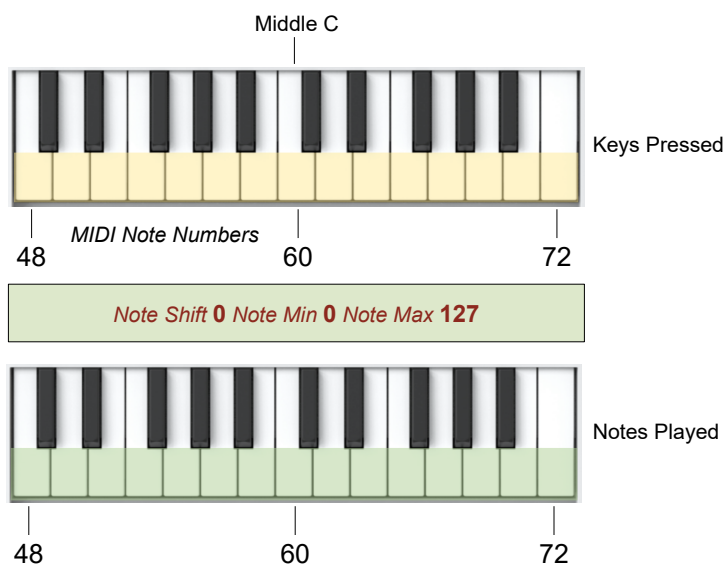
Player Device Common Settings

- **Player Status**
- **Player Level**
- **Player → Delay**
- **Player → Reverb**
- **Note Shift**
- **Note Min**
- **Note Max**
- **Pan Type**
- **Pan Parameter**

Player Keyboard Mappings

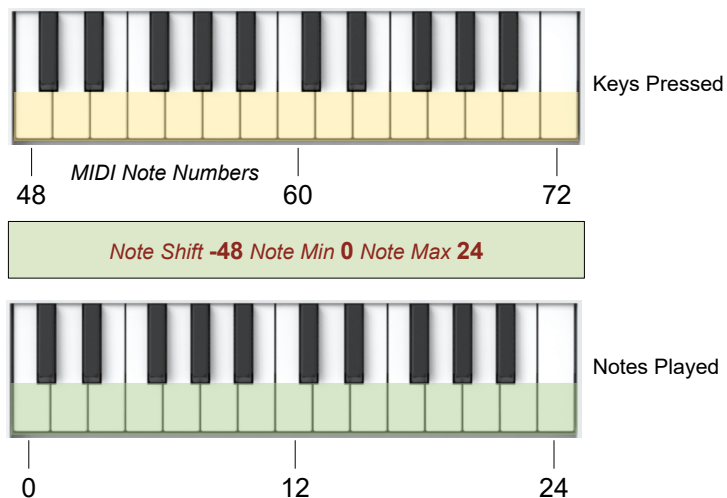
Among the device parameters of each Player Device are 3 settings that determine which keys on the MIDI keyboard will play which notes on the player's designated audio device.

Default Mapping

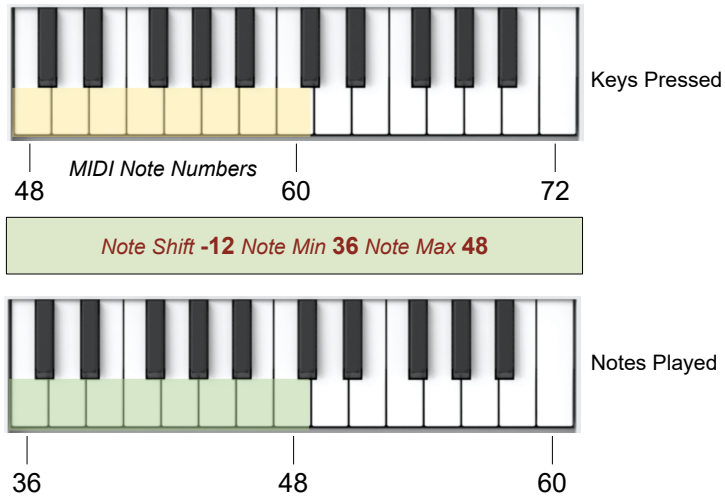


Shifted and Limited Mappings

All Notes Shifted Left 4 Octaves



12 Notes Shifted Left



PitchToMIDI Simulation for WAV and Raw Players

Simultaneous Playback

1. In Ableton, generate MIDI data for the WAV sample in question.
2. Use the MIDI data to create a second WAV sample that plays an instrument using the MIDI data.
3. Output both WAV samples to drive, with similar names.
4. Load the 2 WAV samples onto SD (or Serial Flash) and play the samples simultaneously from the MIDI keyboard to simulate synchronization.

Variations

- Combine both original and MIDI sample in same stereo WAV file (hard LR, or mixed)
- (Best) Add a built-in Pitch to MIDI converter to AudioPlatform

WAV Player

WAV File Player
Teensy4.1 SD WAV Files

 Save Changes  Copy Settings 

Player Status Enabled	Player Level 0.0 dB	Player→Delay 58 %	Player→Reverb 80%
Sound Bank 0	Note Shift -12	Note Min 0	Note Max 99
Pan Type Center	Pan Parameter 63.0	Unassigned 0	Unassigned 0
1	0.5	1	1

Preset 1: Dohee FM1

WavPlyr  RawPlyr  Synth  FmSynth  LineIn 

RAW Player

Raw File Player
Serial Flash Raw Audio Files

 Save Changes  Copy Settings 

Player Status Disabled	Player Level -9.5 dB	Player→Delay 0 %	Player→Reverb 0%
Sound Bank 0	Note Shift 0	Note Min 0	Note Max 99
Pan Type Center	Pan Parameter 63.0	Unassigned 0	Unassigned 0
1	0.5	1	1

Preset 1: Dohee FM1

WavPlyr  RawPlyr  Synth  FmSynth  LineIn 

Basic Synthesizer

Basic Synthesizer

12 Voice Polyphony



Save Changes



Copy Settings



Synth Status Disabled	Synth Level 0.0 dB	Player→Delay 0 %	Player→Reverb 0%
Synth Type AM	Note Shift 0	Note Min 0	Note Max 99
Attack 0 ms	Decay 0 ms	Sustain 0 %	Release 0 ms
Mod Freq 0 Hz	Mod Amt 80 %	Pan Type Center	Pan Parameter 63.0
1	0.5	1	1

Preset 1: Dohee FM1

WavPlyr ■ RawPlyr ■ Synth ■ FmSynth ■ Lineln ■

AudioPlatform for Teensy 4.1

User Guide

Page 35/51

FM Synthesizer

Basic Synthesizer
12 Voice Polyphony

 Save Changes  Copy Settings 

Synth Status
Disabled

Synth Level
-8.5 dB

Player→Delay
20 %

Player→Reverb
60%

 Voices

Voice Name
Lute

Note Shift
14

Note Min
0

Note Max
127

PitchBend Range
1 Semitones

Pan Type
Center

Pan Parameter
63.0

Unassigned

1

0.5

1

1

Preset 1: Dohee FM1

WavPlyr  RawPlyr  Synth  FmSynth  Lineln 

FM Synthesizer Voice Banks

FM Synth Voice Banks
Select Bank and Voice

Voice

BRASS 1

BRASS 2

BRASS 3

STRINGS 1

STRINGS 2

STRINGS 3

ORCHESTRA

PIANO 1

PIANO 2

PIANO 3

E.PIANO 1

GUITAR 1

GUITAR 2

SYN.LEAD 1

BASS 1

BASS 2

E.ORGAN 1

PIPES 1

HARPSICH 1

CLAV 1

VIBE 1

MARIMBA

KOTO

FLUTE 1

ORCH-CHIME

TUB BELLS

STEEL DRUM

TIMPANI

REFS WHISTL

VOICE 1

TRAIN

TAKE OFF

Voice Bank

rom1a

rom1b

rom2a

rom2b

rom3a

rom3b

rom4a

rom4b

Ritch1

Ritch2

User1

User2

User3

User4

User5

User6


FM Synth

FM Synthesizer Emulator

synth_dexed is a port of the Dexed sound engine (<https://github.com/asb2m10/dexed>) as library for the Teensy-3.5/3.6/4.x with an audio shield.

Dexed ist heavily based on <https://github.com/google/music-synthesizer-for-android>

synth_dexed.cpp/h used in AudioPlatform is from:

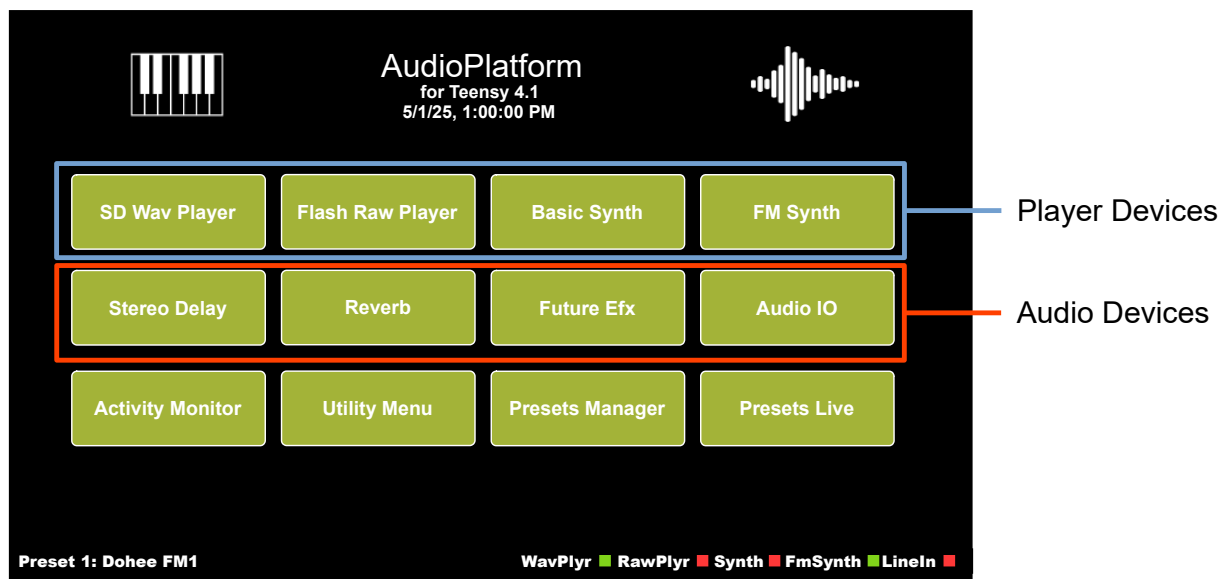
Arduino Resources\Third Party Code\microdexed-touch\third-party\Synth_Dexed\src\

PC/Mac Editor

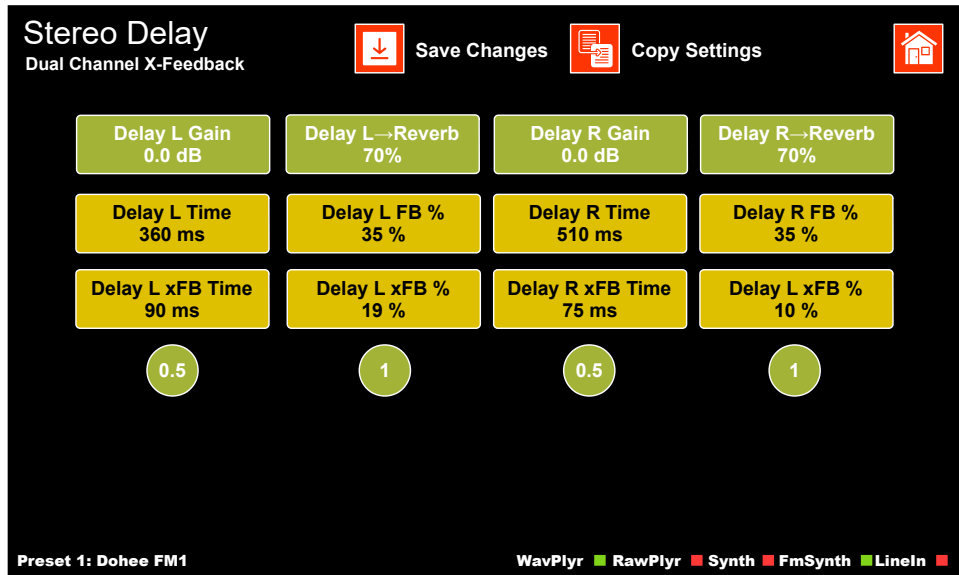
<https://asb2m10.github.io/dexed/>



Audio Device Screens



Stereo Delay

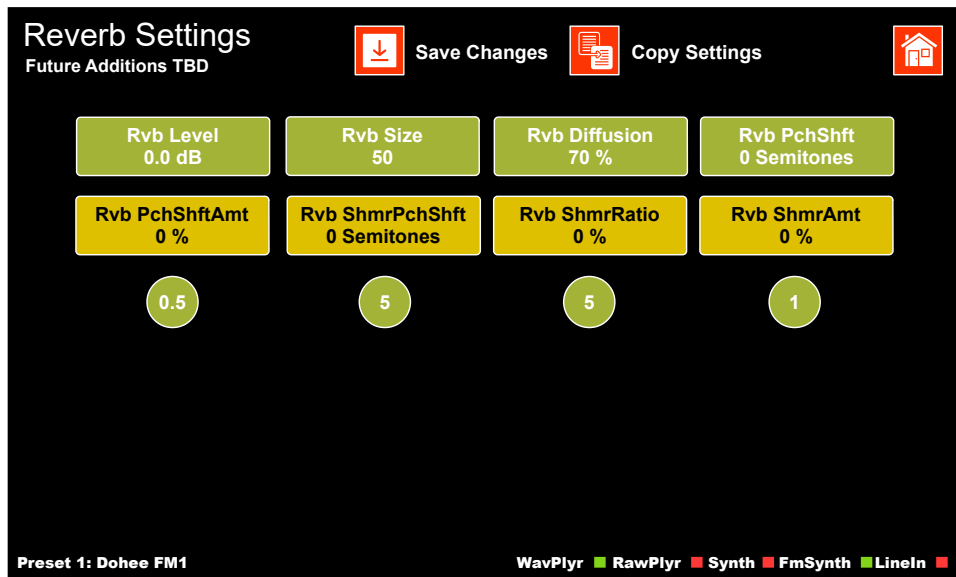


Reverb

Stereo Plate Reverb for Teensy 4 (AudioEffectPlateReverb_i16)

Copyright (c) 2020 by Piotr Zapart

https://github.com/hexeguitar/hexefx_audiolib_i16



HexeFX Plate Reverb Stereo for Teensy4.x

(entries in green set on settings screen)

Main Controls

- **bypass_set** button (bool 0-1) (AP init false)
- **size** - reverb time (0-1)
- **diffusion** - lower values create more echo reflections in the reverb tail (0-1)
- **mix** - dry/wet mixer (default 100% wet)

Filter Controls

- **lodamp** - reverb tail bass loss (0-1) (default 0)
- **hidamp** - reverb tail treble loss (0-1) (default 1)
- **lowpass** - output treble control (0-1) (default 1)
- **hipass** - output bass control (0-1) (default 0)

Freeze Controls

- **freeze** On/Off button for the Freeze (infinite reverb) mode (bool) (default 0)
- **freezeBleedIn** - controls how much of the input signal is fed into the reverb tank while in Freeze mode (0-1) (default 0)

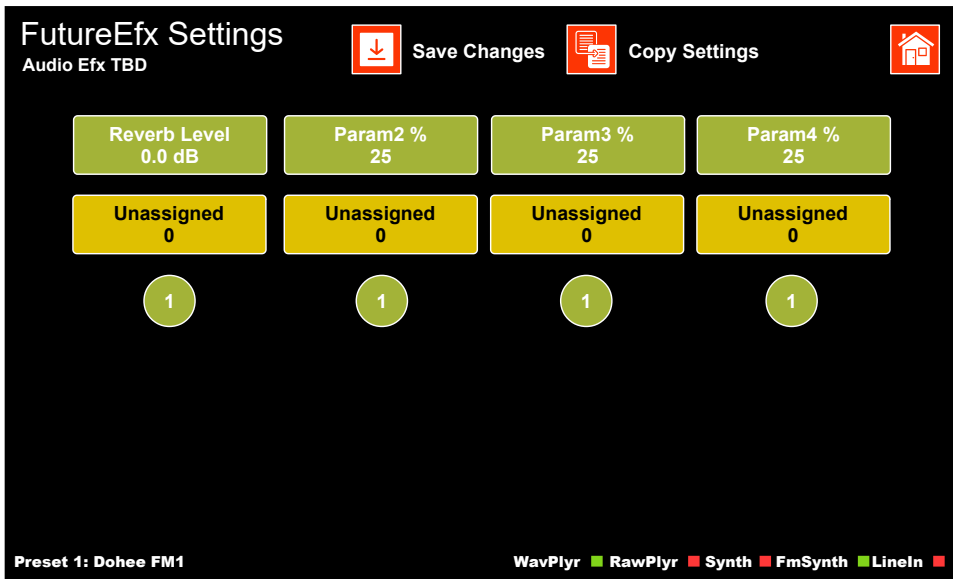
Pitch Controls

- **pitchSemitones** - main pitch shifter applied to the whole reverb sound, semitones (int8_t -12,+24) **(default 0)**
- **pitchMix** - main pitch shifter dry/wet mixer (0-1) **(default 0)**

Shimmer Controls

- **shimmer** - amount of shimmer effect applied to the reverb tail (0-1) **(default 0)**
- **shimmerPitch** - pitch ratio setting for the Shimmer effect (0-1). **(default 0)**
- **shimmerPitchSemitones** - Sets the shimmer effect pitch in semitones (-12, +24) **(default 0)**

Future Efx



Audio IO

Audio IO Settings

Audio In/Out/Codec

 Save Changes

 Copy Settings



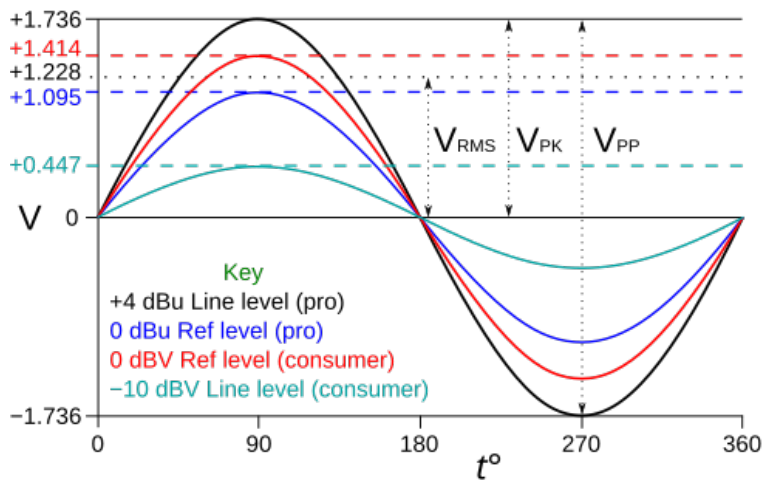
LineIn Status Disabled	LineIn Level 1.33 Volts p-p	LineOut Level 2.83 Volts p-p	Phone Volume 80 %
LineIn→Delay 0 %	LineIn→Reverb 100 %	Unassigned 0	Unassigned 0
1	1	1	1

Preset 1: Dohee FM1

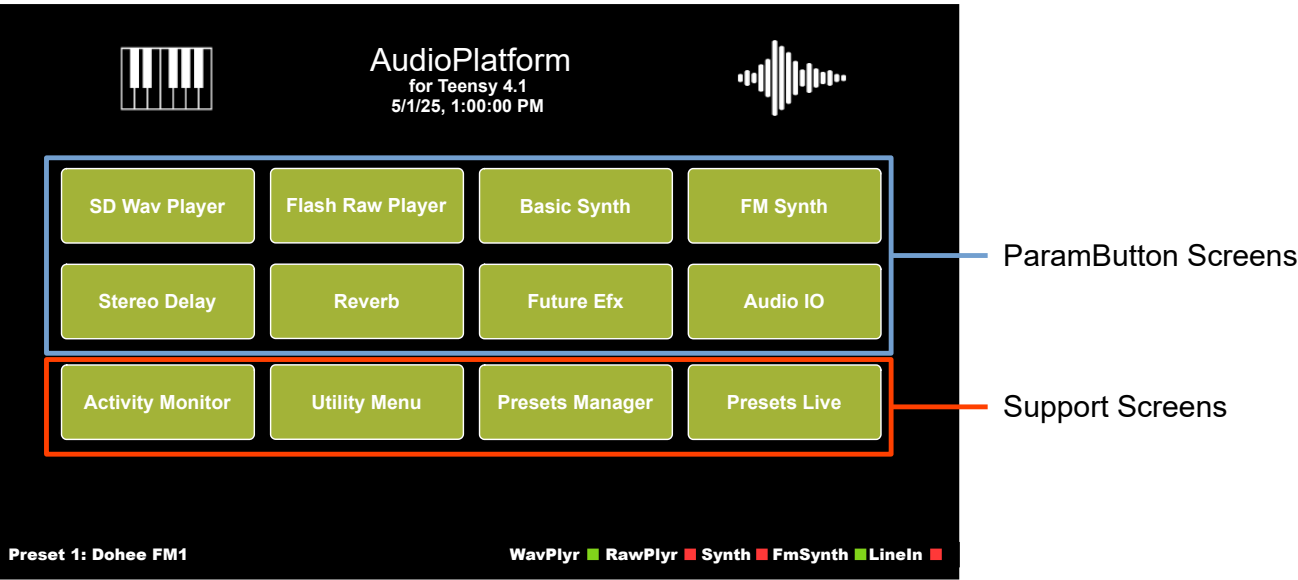
WavPlyr ■ RawPlyr ■ Synth ■ FmSynth ■ LineIn ■

Line Input Level

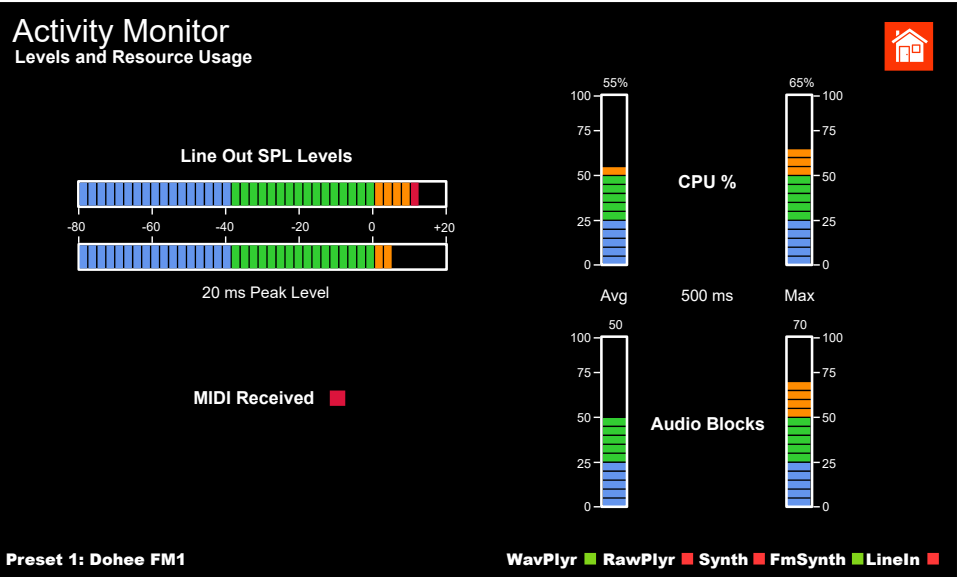
Rotary seems backward, but is correct.



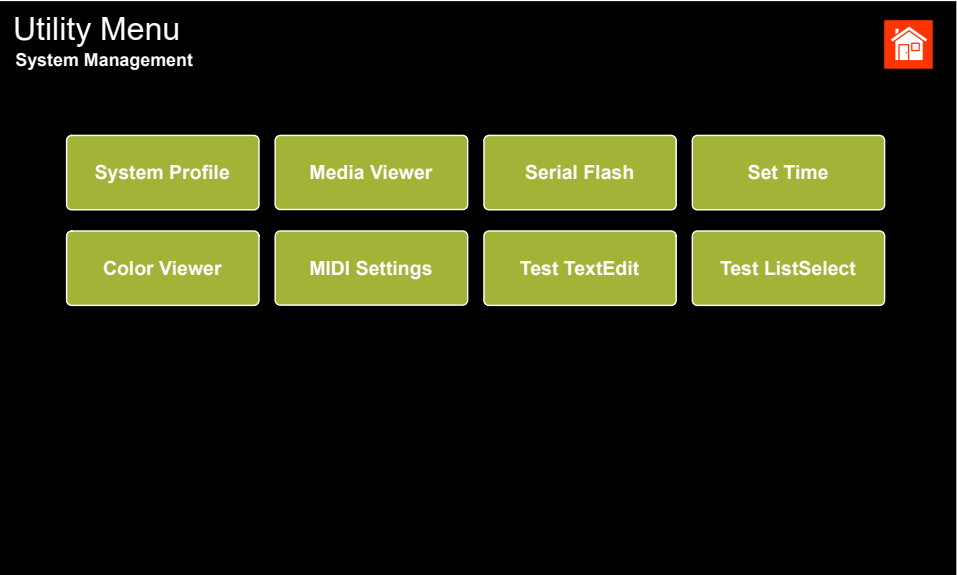
Supprt Screens



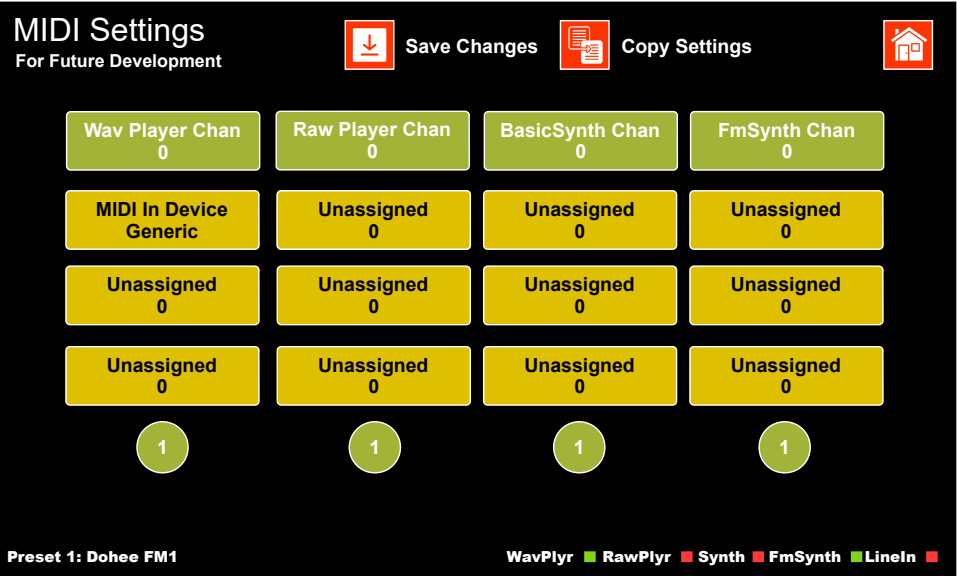
Activity Monitor



Utility Menu



MIDI Settings



Presets Manager

Presets Management

Load,Save,Edit Presets & Banks



Dohee FM1	Synth 1	Synth 2	Synth Rpt	Dohee Efx	Synth Rvb	Synth Rich	Woo Woo
1	2	3	4	5	6	7	8
MicroTone L	Synth Pulse	MicroTone S	Synth Marim	WAV Plyr	RAW Plyr	Basic Synth	FM Cluster
9	10	11	12	13	14	15	16
FM Synth	Dohee FM2	Dohee Shift	Multi 1		Synth Rpt 2	Guitar	
17	18	19	20	21	22	23	24
Synth Pulse	Cluster	FM Piano					ALL OFF
25	26	27	28	29	30	31	32

Preset: MicroTone L

Active Bank: DefaultBank

Press Load, Save, Import, Clear, or Rename

Press Load or Save

LoadSaveImportClearRenameCancel

LoadSaveDefaultSD CardCancel

Presets Management

Load,Save,Edit Presets & Banks



Dohee FM1	Synth 1	Synth 2	Synth Rpt	Dohee Efx	Synth Rvb	Synth Rich	Woo Woo
1	2	3	4	5	6	7	8
MicroTone L	Synth Pulse	MicroTone S	Synth Marim	WAV Plyr	RAW Plyr	Basic Synth	FM Cluster
9	10	11	12	13	14	15	16
FM Synth	Dohee FM2	Dohee Shift	Multi 1		Synth Rpt 2	Guitar	
17	18	19	20	21	22	23	24
Synth Pulse	Cluster	FM Piano					ALL OFF
25	26	27	28	29	30	31	32

Preset: MicroTone L

Active Bank: DefaultBank

Press Load, Save, Import, Clear, or Rename

Press Load or Save

LoadSaveImportClearRenameCancel

LoadSaveDefaultSD CardCancel

Presets Live

Presets Live

Direct Preset & Bank Access

All Notes Off
 Ignore Note Off

1 Dohee FM1	2 Synth 1	3 Synth 2	4 Synth Rpt	DefaultBank.pbk DefaultBank-250220.pbk DefaultBank-250319.pbk DefaultBank-250415.pbk DefaultBank-250519.pbk ImportActiveTest.pbk SynthMicroToneCmnDly.pbk FmSynthPanTest.pbk MultiPlayerTest1.pbk MultiPlayerTest2.pbk WavPlayerMultiEffects.pbk BasicSynthMultiEffects.pbk
5 Dohee Efx	6 Synth Rvb	7 Synth Rich	8 Woo Woo	
9 MicroTone L	10 Synth Pulse	11 MicroTone S	12 Synth Marim	
13 WAV Plyr	14 Raw Plyr	15 Basic Synth	16 FM Cluster	
17 FM Synth	18 Dohee FM2	19 Dohee Shift	20 Multi 1	
21	22 Synth Rpt 2	23 Guitar	24	
25 Synth Pulse	26 Cluster	27 FM Piano	28	
29	30	31	32 ALL OFF	

Active Bank: DefaultBank.pbk

Preset Banks on SD

Preset 1: Dohee FM1

WavPlyr ■ RawPlyr ■ Synth ■ FmSynth ■ LineIn ■

Rotary Encoders

Analog Potentiometers vs Rotary Encoders

The Teensy Audio Tutorial Adapter includes 3 potentiometer knobs that are monitored through the 3 of the Teensy's Analog Input ports, which utilize 10-bit ADCs for a range of 0-1023. It is in the nature of devices of this sort (as well as sensors of various types) when connected directly to be subject to a good deal of noise and "jitter", requiring such techniques as input averaging, filtering, and such to achieve stability.

In addition, there is the issue of matching the ADCs input range to the range of the parameter to be controlled, generally by scaling or mapping one range to another. While this can work acceptably when the input range is much larger than that of the target--- for example, mapping 0-1023 to a target range of 0-100 allows for 10 values of the input to be mapped to a single target value, resulting in much greater stability---though not always.

However, when the target range is close to or exceeds the input range, the use of analog inputs becomes problematic (this is to ignore that fact that potentiometers are often non-linear in their response). For example, in the case of a digital delay of maximum time 1 second, if one wishes to set to an exact 1 millisecond value, the task becomes nearly impossible with a 10-bit converter, and even more so for a 2 second digital delay with that granularity.

In such cases, a very good and precise alternative is provided by a Rotary Encoder, which has the appearance of an analog knob, but rather than outputting voltages, it monitors "clicks" as it is turned clockwise or counter-clockwise, without the limit of stop points. In use, it can increment or decrement a numeric value by adding or subtracting the number of clicks from the current value.

So, the encoder can be used to set very precise values of the parameter to which it is assigned, regardless of range. But, with a range of 1000 or more points, this would be quite tedious if the value could only be changed by +/- 1 per click. Fortunately, many such devices also include a pushbutton function for the knob, such that, with successive pushes, the increment value can be moved through a range of values such as (1, 5, 10, 20) or (1, 10, 50, 100, 500), etc. depending on the situation.

The KY-050 encoders used in this project are very common, and obtainable from many online sources, including ProtoSupplies. In addition, there are numerous online guides to how encoders work and how to use them. For this project, a class "Rotary Encoder" was designed, based on the Arduino open-source library:

<https://github.com/luni64/EncoderTool>

Appendix

References

AudioPlatform Source and Documentation

[AudioPlatform Source Code](#)

[AudioPlatform Documentation](#)

ProtoSupplies.com

[Web Site](#)

[Prototyping System For Teensy 4.1](#)

[Teensy 4.1 Fully Loaded For Prototyping System](#)

[Project System For Teensy 4.1](#)

[Teensy 4.x Audio Adapter](#)

PJRC / Teensy

[PJRC](#)

[Teensy User Forum](#)

[Teensy 4.1](#)

[Teensyduino](#)

[Teensyduino Libraries](#)

[Teensy Audio Shield](#)

[Teensy Audio Tutorial](#)

Teensy DIY Audio Projects and Kits

[Teensy Forum](#)

[TeensyMix Synth - DIY 8 Voice Synthesizer](#)

[Shruti II Teensy 4.1 DIY Synthesizer](#)

[Polyhonic Teensy DIY Sampler](#)

[Teensy Super Audio Board](#)

[MicroDexed Touch Synthesizer](#)

[Polyhonic Teensy DIY Sampler](#)

YouTube

[DIY Dirtywave M8 headless using a Teensy 4.1](#)

[DX7 Clone on Teensy 4.0](#)

[Electrotechnique TSynth - Teensy 4.1 based DIY Synthesizer](#)

[Teensy 4.1 Drone Synth](#)

[Teensy MIDI Step Sequencer](#)

[FALA Synthesizer - 8 Voice DIY Teensy Synth](#)



<https://fonts.google.com/>



Appendix

References

AudioPlatform Source and Documentation

[AudioPlatform Source Code](#)

[AudioPlatform Documentation](#)

ProtoSupplies

[ProtoSupplies Web Site](#)

[Prototyping System For Teensy 4.1](#)

[Teensy 4.1 Fully Loaded For Prototyping System](#)

[Project System For Teensy 4.1](#)

[Teensy 4.x Audio Adapter](#)

PJRC / Teensy

[PJRC](#)

[Teensy User Forum](#)

[Teensy 4.1](#)

[TeensyDuino](#)

[TeensyDuino Libraries](#)

[Teensy Audio Shield](#)

[Teensy Audio Tutorial](#)

Teensy DIY Audio Projects and Kits

Teensy Forum

[TeensyMix Synth - DIY 8 Voice Synthesizer](#)

[Shruti II Teensy 4.1 DIY Synthesizer](#)

[Polyhonic Teensy DIY Sampler](#)

[Teensy Super Audio Board](#)

[MicroDexed Touch Synthesizer](#)

[Polyhonic Teensy DIY Sampler](#)

YouTube

[DIY Dirtywave M8 headless using a Teensy 4.1](#)

[DX7 Clone on Teensy 4.0](#)

[Electrotechnique TSynth - Teensy 4.1 based DIY Synthesizer](#)

[Teensy 4.1 Drone Synth](#)

[Teensy Midi Step Sequencer](#)

[FALA Synthesizer - 8 Voice DIY Teensy Synth](#)